Page 1 — Introduction to Agentic AI

Agentic AI systems are applications that can plan, reason about tasks, use tools, and produce useful outcomes with limited human intervention.

A typical agentic application contains an orchestrator, specialized services or tools, a language model, and application data. The orchestrator decides which capability should be invoked and in what order.

For example, an email assistant may use a Gmail service to retrieve messages and an OpenAI service to summarize them. A travel assistant might use weather, maps, flight, and hotel tools.

The most important engineering principle is separation of responsibilities. Each component should have a clear purpose so that the system is easier to test, maintain, and extend.

Page 2 — Retrieval-Augmented Generation

Retrieval-Augmented Generation, commonly called RAG, allows an application to provide an LLM with information retrieved from an external knowledge source.

A basic RAG pipeline has two major stages. During ingestion, documents are loaded, split into chunks, converted into embeddings, and stored in a vector database. During question answering, the user's question is embedded and compared with stored vectors.

The most relevant chunks are retrieved and placed into the prompt sent to the language model. The model then generates an answer using the retrieved context.

RAG is useful when information is private, frequently changing, domain-specific, or too large to place directly into every prompt.

Page 3 — Document Chunking

Chunking is the process of splitting a document into smaller pieces before creating embeddings. The objective is to create chunks that are large enough to preserve meaning but small enough to retrieve precisely.

A naive chunking strategy can split text into a fixed number of characters. For example, a document can be divided into chunks of 1,000 characters with an overlap of 100 characters.

Overlap helps preserve context when an important sentence occurs near a chunk boundary. Without overlap, related information may be separated into different chunks.

Chunk size is an important parameter in RAG quality. Very small chunks may lose context, while very large chunks may retrieve too much irrelevant information.

Page 4 — Embeddings and Vector Databases

An embedding is a numerical representation of text. An embedding model converts a sentence, paragraph, or document chunk into a vector containing many numerical dimensions.

Texts with similar meanings tend to have embeddings that are close together according to a similarity metric. This allows an application to search for semantic similarity rather than relying only on exact keyword matches.

A vector database stores embeddings together with associated text and metadata. During retrieval, the application provides a query vector and asks the database for the most similar stored vectors.

Common metadata includes the document name, page number, chunk number, and source location. Metadata can later be used for filtering and citations.

Page 5 — End-to-End Naive RAG

A simple RAG application can be implemented as a sequence of independent services: document loader, chunking service, embedding service, vector store, retrieval service, and generation service.

The ingestion flow is: load document, extract text, create chunks, generate embeddings, and store the chunks and vectors. This process normally happens before users ask questions.

The query flow is: receive question, generate a query embedding, retrieve the top relevant chunks, construct a prompt containing the question and retrieved context, and call the language model.

For learning purposes, a naive RAG should avoid hiding these steps behind a framework. Understanding the individual operations makes it easier to understand more advanced RAG frameworks later.

Useful experiments include changing chunk size, changing overlap, retrieving different numbers of chunks, and asking questions whose answers appear on different pages.